Lab_1sagar

July 25, 2025

0.1 Section A: Line and Scatter Plots

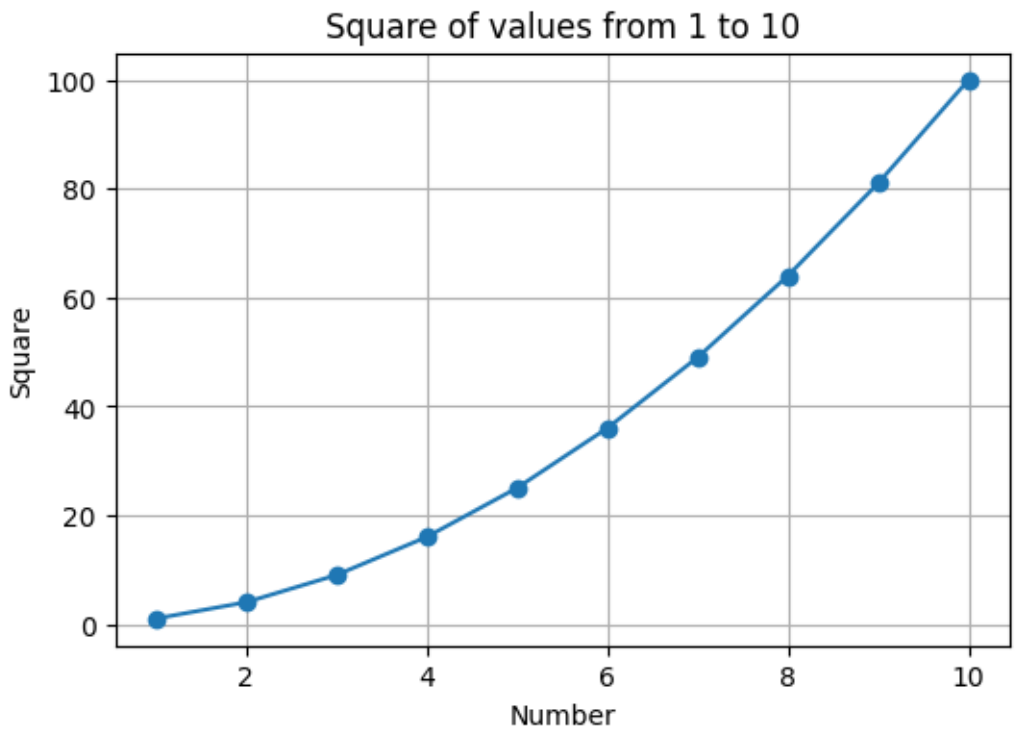
[30]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets

from ipywidgets import interact
```

[28]:
```
!pip install mplfinance
```

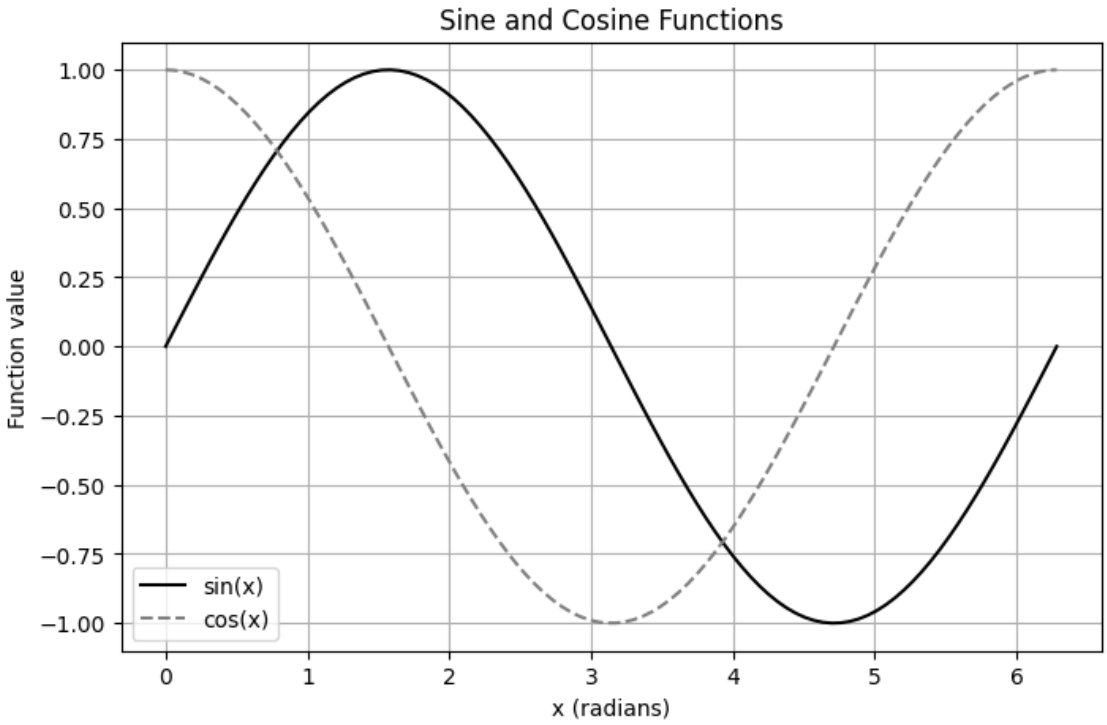
```
Collecting mplfinance
  Downloading mplfinance-0.12.10b0-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-
packages (from mplfinance) (3.10.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages
(from mplfinance) (2.2.2)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->mplfinance) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-
packages (from matplotlib->mplfinance) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->mplfinance) (4.58.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->mplfinance) (1.4.8)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.11/dist-
packages (from matplotlib->mplfinance) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->mplfinance) (24.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-
packages (from matplotlib->mplfinance) (11.2.1)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->mplfinance) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->mplfinance)
(2.9.0.post0)
```

```
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-
packages (from pandas->mplfinance) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-
packages (from pandas->mplfinance) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-
packages (from python-dateutil>=2.7->matplotlib->mplfinance) (1.17.0)
Downloading mplfinance-0.12.10b0-py3-none-any.whl (75 kB)
                         75.0/75.0 kB
2.8 MB/s eta 0:00:00
Installing collected packages: mplfinance
Successfully installed mplfinance-0.12.10b0
```

[ ]:
```python
# 1. Create a line plot showing the square of values from 1 to 10
x = np.arange(1, 11)
y = x ** 2

plt.figure(figsize=(6, 4))
plt.plot(x, y, marker='o')
plt.title('Square of values from 1 to 10')
plt.xlabel('Number')
plt.ylabel('Square')
plt.grid(True)
plt.show()
```

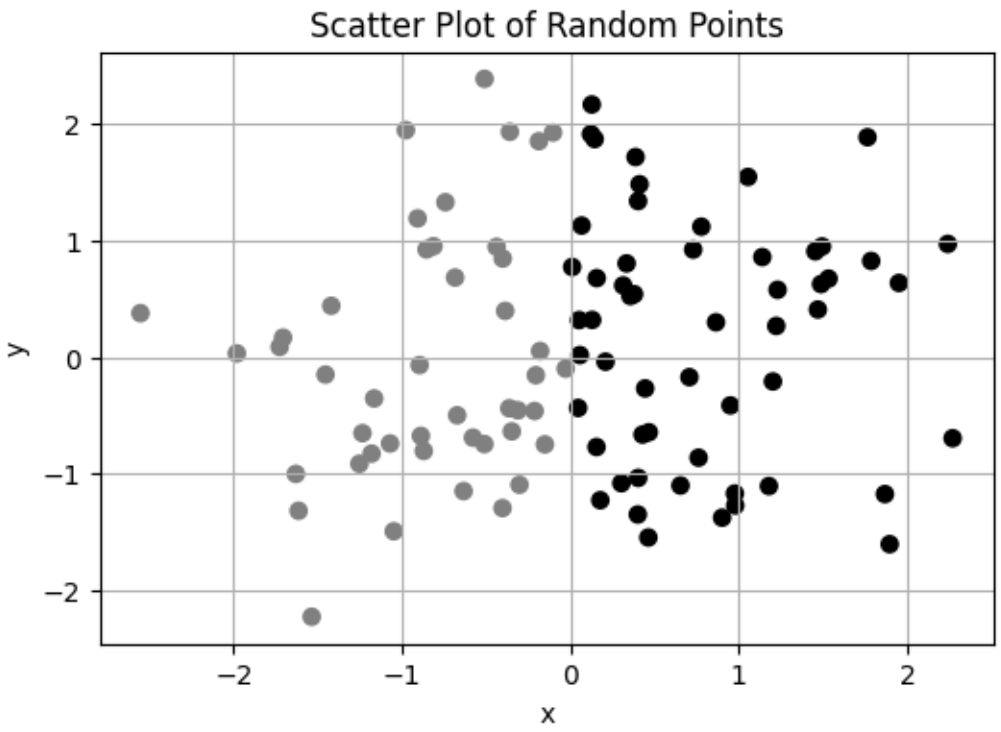


[13]:
```
# 2. Generate 100 values between 0 and 2 . Plot sin(x) and cos(x) on the same␣
 ↪figure.
x = np.linspace(0, 2 * np.pi, 100)
sin_y = np.sin(x)
cos_y = np.cos(x)

plt.figure(figsize=(8, 5))
plt.plot(x, sin_y, label='sin(x)', color='black', linestyle='-')        # Solid␣
 ↪black
plt.plot(x, cos_y, label='cos(x)', color='gray', linestyle='--')        # Dashed␣
 ↪gray
plt.title('Sine and Cosine Functions')
plt.xlabel('x (radians)')
plt.ylabel('Function value')
plt.legend()
plt.grid(True)
plt.show()
```



[14]:
```
# 3. Scatter Plot with Random Data
np.random.seed(0)
x = np.random.randn(100)
y = np.random.randn(100)
```

```
# Use two shades of gray
colors = ['black' if val > 0 else 'gray' for val in x]

plt.figure(figsize=(6, 4))
plt.scatter(x, y, c=colors)
plt.title('Scatter Plot of Random Points')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.show()
```



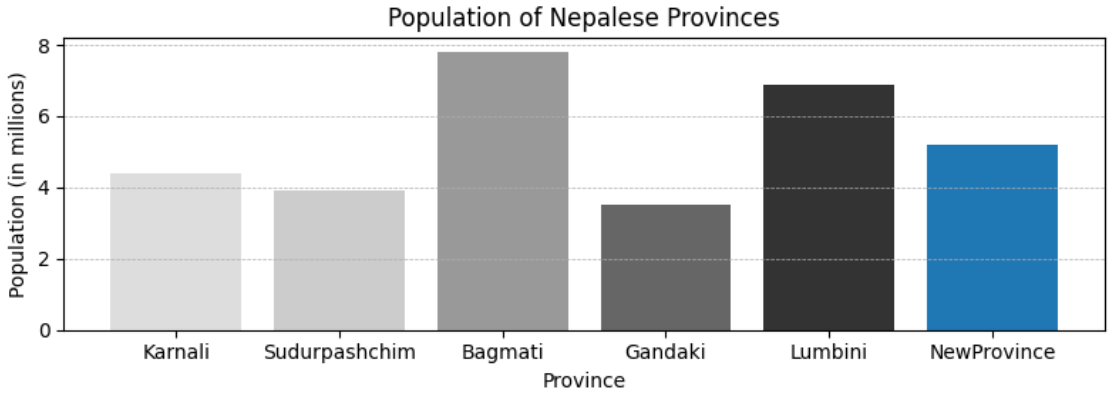
0.2 Section B: Bar and Pie Charts

Bar Chart – Province Populations - Use dummy data for the population of five provinces in Nepal. g

- Create a bar chart.
- Label each bar with the province name and set axis titles.
- Use a unique color for each bar.

[20]:
```
provinces = ['Karnali', 'Sudurpashchim', 'Bagmati', 'Gandaki', 'Lumbini',␣
↪'NewProvince']
populations = [4.4, 3.9, 7.8, 3.5, 6.9, 5.2]
```

```
colors = ['#eeeeee', '#cccccc', '#999999', '#666666', '#333333']

provinces = ['Karnali', 'Sudurpashchim', 'Bagmati', 'Gandaki', 'Lumbini',␣
 ↪'NewProvince']
populations = [4.4, 3.9, 7.8, 3.5, 6.9, 5.2]

colors = ['#dddddd', '#cccccc', '#999999', '#666666', '#333333', '#1f77b4']

plt.figure(figsize=(8, 3))
bars = plt.bar(provinces, populations, color=colors)
plt.title("Population of Nepalese Provinces")
plt.xlabel("Province")
plt.ylabel("Population (in millions)")
plt.grid(axis='y', linestyle='--', linewidth=0.5)
plt.tight_layout()
plt.show()
```



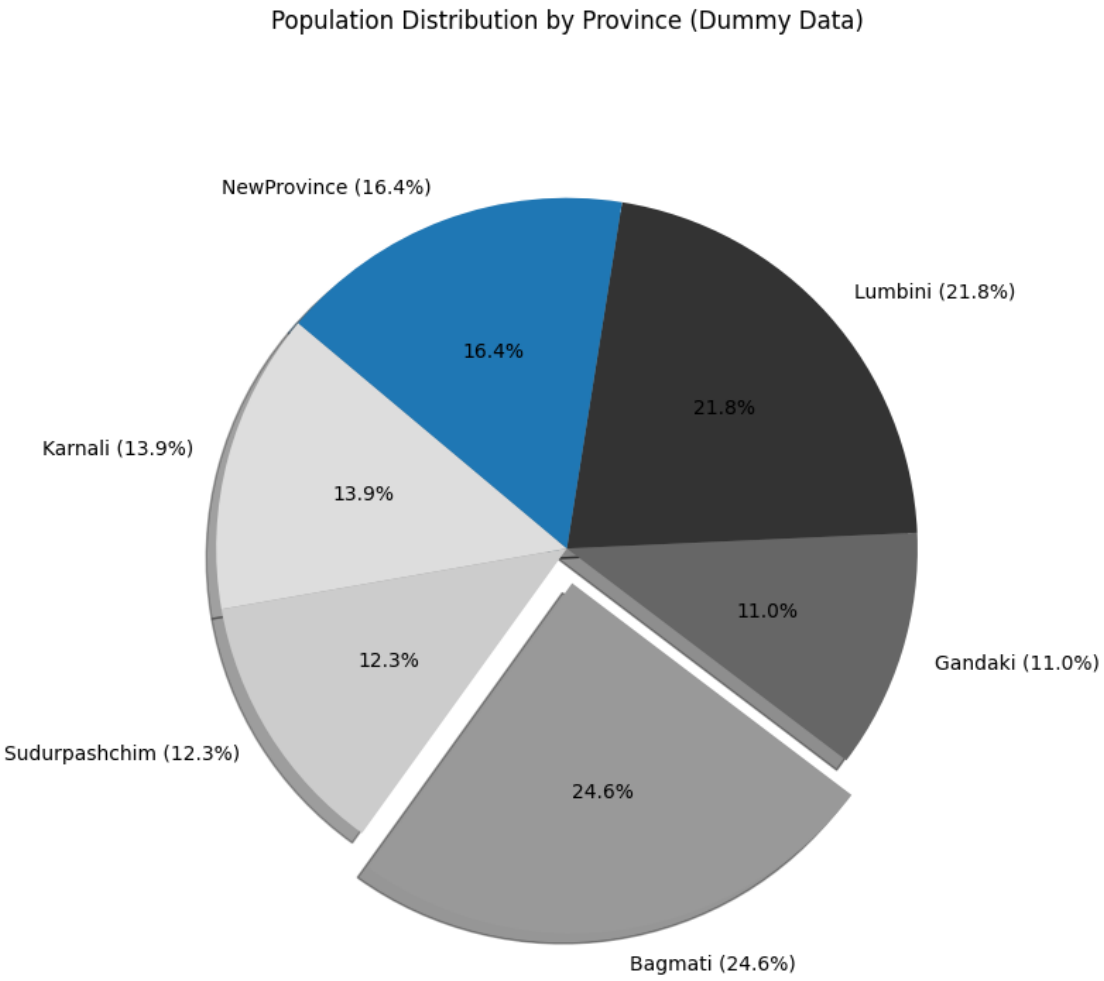
0.2.1 Pie Chart – Province Populations

Using the same data as above:

Create a pie chart. - Label each slice with the province name and percentage. - Explode the largest slice for emphasis.

[21]:
```
provinces = ['Karnali', 'Sudurpashchim', 'Bagmati', 'Gandaki', 'Lumbini',␣
 ↪'NewProvince']
populations = [4.4, 3.9, 7.8, 3.5, 6.9, 5.2]

colors = ['#dddddd', '#cccccc', '#999999', '#666666', '#333333', '#1f77b4']

total = sum(populations)
percentages = [p / total * 100 for p in populations]
```

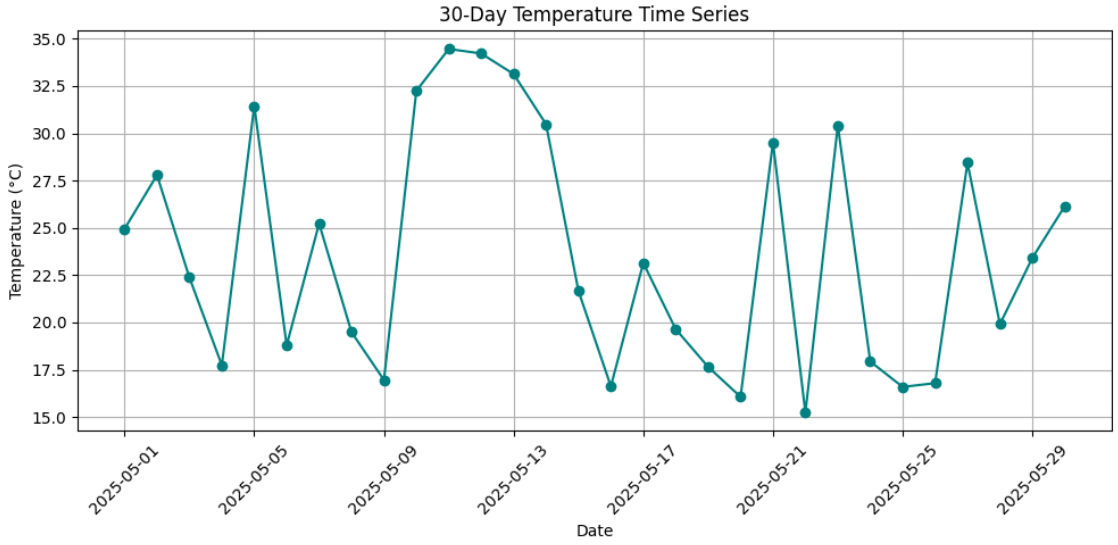
```
# Explode the largest slice Bagmati (index 2)
explode = [0, 0, 0.1, 0, 0, 0]

plt.figure(figsize=(8, 8))
plt.pie(populations, labels=[f'{prov} ({p:.1f}%)' for prov, p in zip(provinces,␣
 ↪percentages)],
        colors=colors, autopct='%.1f%%', startangle=140, shadow=True,␣
 ↪explode=explode)

plt.title('Population Distribution by Province (Dummy Data)')
plt.axis('equal')  # Make it a circle
plt.tight_layout()
plt.show()
```



0.3 Section C: Time Series Plotting

6. **Temperature Time Series**
   - Create a `pandas` DataFrame with a `Date` column (30 days) and random temperature values.
   - Plot the temperature as a line chart.
   - Format the x-axis to show dates properly.

[ ]:
```python
# 1. Create a date range for 30 days
date_range = pd.date_range(start="2025-05-01", periods=30, freq='D')

# 2. Generate random temp data
temperature = np.random.uniform(low=15, high=35, size=30)

# 3. Create a DataFrame
df = pd.DataFrame({
    'Date': date_range,
    'Temperature': temperature
})

# 4. Plot the temperature time series
plt.figure(figsize=(10, 5))
plt.plot(df['Date'], df['Temperature'], marker='o', linestyle='-', color='teal')
plt.title("30-Day Temperature Time Series")
plt.xlabel("Date")
plt.ylabel("Temperature (°C)")

# 5. Format the x-axis
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.show()
```

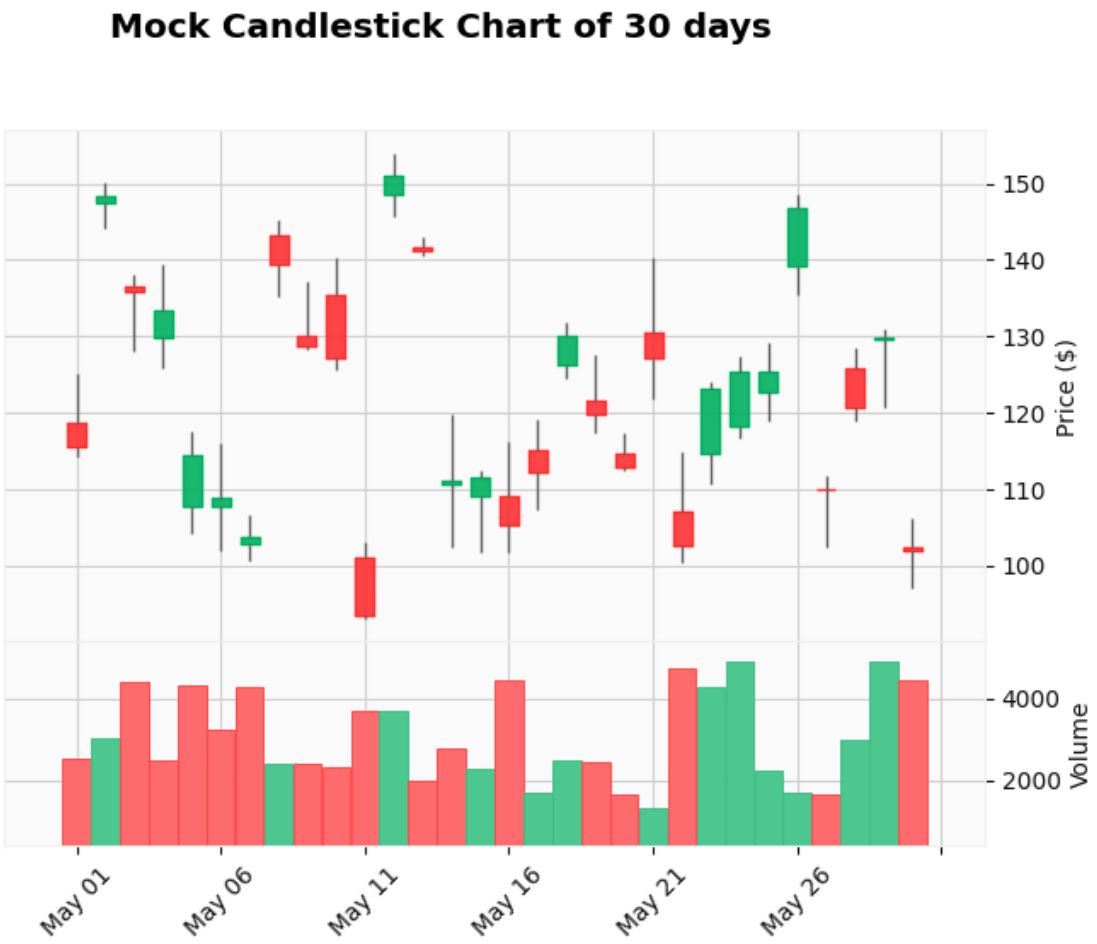


0.4 Section D: Candlestick Chart

7. **Candlestick Chart with `mplfinance`**
   - Create a DataFrame with columns: `Date`, `Open`, `High`, `Low`, `Close`, and optionally `Volume`.
   - Plot a candlestick chart using `mplfinance`.
   - Customize the title and figure size.

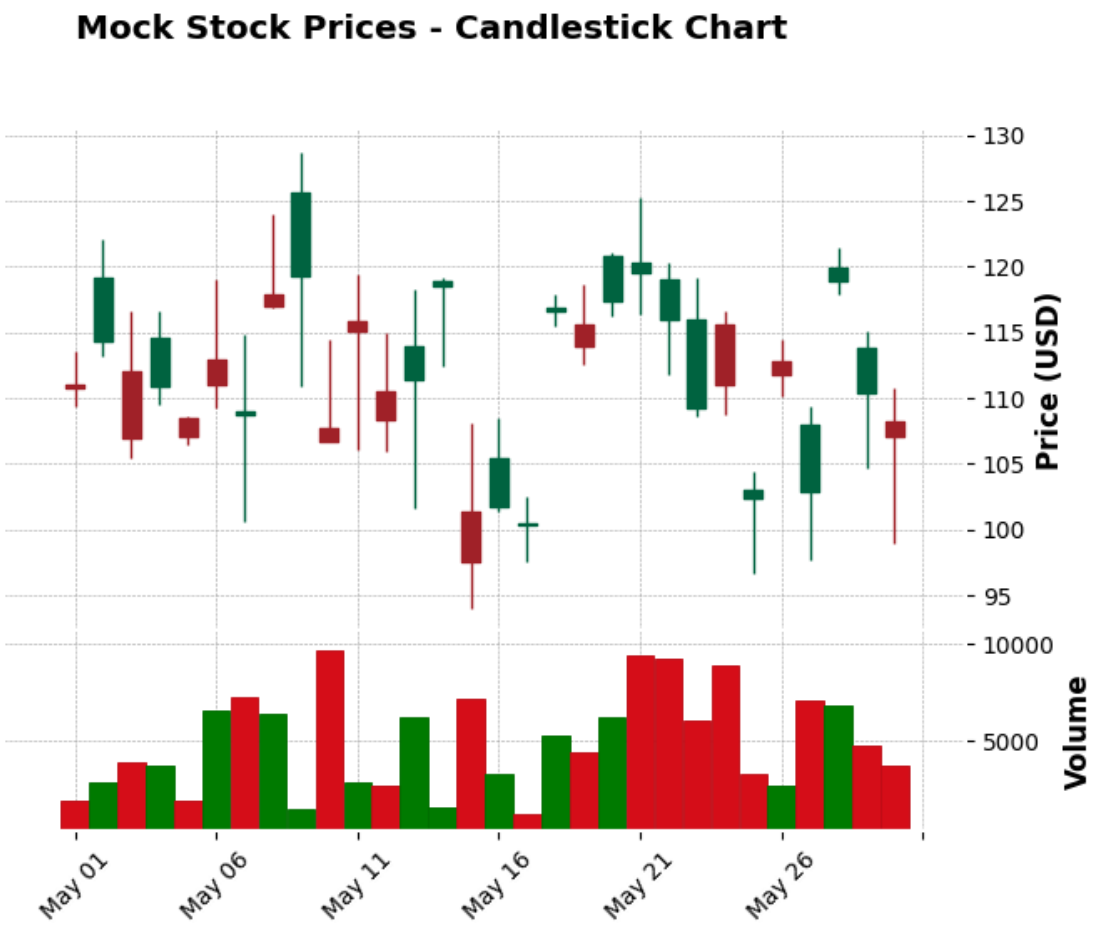
[29]:
```python
import mplfinance as mpf

# 1. Generate a date range
date_range = pd.date_range(start="2025-05-01", periods=30, freq='D')

# 2. Generate mock stock data
np.random.seed(42)
open_prices = np.random.uniform(100, 150, size=30)
high_prices = open_prices + np.random.uniform(1, 10, size=30)
low_prices = open_prices - np.random.uniform(1, 10, size=30)
close_prices = np.random.uniform(low_prices, high_prices)
volume = np.random.randint(1000, 5000, size=30)

# 3. Create the DataFrame
df = pd.DataFrame({
    'Date': date_range,
    'Open': open_prices,
    'High': high_prices,
    'Low': low_prices,
    'Close': close_prices,
    'Volume': volume
```

```
})
df.set_index('Date', inplace=True)

# 4. Plot candlestick chart
mpf.plot(
    df,
    type='candle',
    style='yahoo',
    title='Mock Candlestick Chart of 30 days',
    ylabel='Price ($)',
    volume=True,
    figsize=(8, 6)
)
```



[ ]:
```
import mplfinance as mpf
import pandas as pd
import numpy as np
```

```
# 1. Create 30 days of date data
date_range = pd.date_range(start='2025-05-01', periods=30, freq='D')

# 2. Generate realistic stock price data
np.random.seed(0)
open_prices = np.random.uniform(100, 120, size=30)
high_prices = open_prices + np.random.uniform(0, 10, size=30)
low_prices = open_prices - np.random.uniform(0, 10, size=30)
close_prices = np.random.uniform(low_prices, high_prices)
volumes = np.random.randint(1000, 10000, size=30)

# 3. Create a DataFrame
df = pd.DataFrame({
    'Date': date_range,
    'Open': open_prices,
    'High': high_prices,
    'Low': low_prices,
    'Close': close_prices,
    'Volume': volumes
})

# 4. Set the Date as the index
df.set_index('Date', inplace=True)

# 5. Plot the candlestick chart
mpf.plot(
    df,
    type='candle',
    style='charles',  # you can try 'yahoo', 'mike', etc.
    title='Mock Stock Prices - Candlestick Chart',
    ylabel='Price (USD)',
    ylabel_lower='Volume',
    volume=True,
    figsize=(8, 6)
)
```



0.5 Section E: Interactivity

8. **Interactive Function Plotter**
   - Use `ipywidgets` to let a user choose:
     - Checkbox to show `sin`, `cos`, `tan` curves
     - A color
     - Number of points (e.g., 50, 100, 200)
   - Plot the selected function interactively with the chosen parameters.

[31]:
```
# Interactive plotting function without points slider
def plot_trig(show_sin, show_cos, show_tan, color):
    x = np.linspace(0, 2 * np.pi, 200)  # Fixed number of points

    plt.figure(figsize=(10, 6))

    if show_sin:
        plt.plot(x, np.sin(x), label='sin(x)', color=color)
    if show_cos:
```

```
        plt.plot(x, np.cos(x), label='cos(x)', linestyle='--', color=color)
    if show_tan:
        y = np.tan(x)
        y[np.abs(y) > 10] = np.nan  # avoid large jumps in tan
        plt.plot(x, y, label='tan(x)', linestyle=':', color=color)

    plt.ylim(-2, 2)
    plt.title("Interactive Trig Function Plotter")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.legend()
    plt.grid(True)
    plt.show()

# Widgets (without points)
interact(
    plot_trig,
    show_sin=widgets.Checkbox(value=True, description='sin(x)'),
    show_cos=widgets.Checkbox(value=False, description='cos(x)'),
    show_tan=widgets.Checkbox(value=False, description='tan(x)'),
    color=widgets.Dropdown(options=['blue', 'green', 'red', 'purple'],␣
 ↪value='blue')
)
```

[31]